Rapport d'avancement – Implémentation EMG-VBA pour la résolution de problèmes inverses par modèles de diffusion

Gatien Music Music Seguy
M2 FESup/SIEN – ENS Paris-Saclay
Encadrant : Thomas Rodet (SATIE/CNRS)

Avril 2026

Table des matières

1 Introduction **3**

2 Architecture du projet **3**
2.1 Structure modulaire . . . 3
2.2 Interface opérateur générique . . . 3
2.3 Compatibilité multi-modèles . . . 4

3 Passage CPU → GPU **4**
3.1 Motivation . . . 4
3.2 Difficultés rencontrées . . . 4

4 Pseudo-code de l'algorithme **5**

5 Équations clés **5**
5.1 Mise à jour des précisions (éq. 75–85) . . . 5
5.2 Distribution de référence (éq. 94–95) . . . 6
5.3 Mise à jour de $q(x_0)$ (éq. 115–116, cas $s_2 = 0$) . . . 6
5.4 Énergie libre négative (éq. 121–123) . . . 6

6 Premiers résultats (EMG-VBA seul) **6**
6.1 Configuration expérimentale . . . 6
6.2 Reconstruction . . . 6
6.3 Convergence de l'énergie libre . . . 7
6.4 Estimation automatique des variances . . . 7

7 Protocole expérimental de comparaison **7**
7.1 Méthodes comparées 7
7.2 Protocole 7

8 Résultats du benchmark **8**
8.1 Tableau récapitulatif 8
8.2 Analyse 8

9 Perspectives et idées **9**
9.1 Transport optimal pour l'initialisation inter-pas 9
9.2 Sous-espace de dimension 2 9
9.3 Preuves de convergence 9
9.4 Autres opérateurs de dégradation 10
9.5 Robustesse au niveau de bruit 10
9.6 Module Python 10

1 Introduction

Ce rapport présente l'implémentation et les premiers résultats expérimentaux de l'algorithme EMG-VBA (Expected Maximization Gradient – Variational Bayesian Approximation) appliqué à la résolution de problèmes inverses en imagerie, couplé à un modèle de diffusion DDPM comme a priori génératif. L'ensemble de la théorie (modèle probabiliste, dérivation des mises à jour, calcul du pas sous-optimal, énergie libre) est détaillé dans le rapport théorique [1]. Le présent document se concentre sur l'architecture logicielle, les choix d'implémentation, les résultats obtenus, et la comparaison avec les méthodes de l'état de l'art (DPS, ΠGDM).

Le problème inverse considéré est :

$$\boldsymbol{y} = \mathcal{A}\boldsymbol{x}_0^* + \sigma_b \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, I_m) \tag{1}$$

où $\mathcal{A}$ est un opérateur linéaire de dégradation (flou, sous-échantillonnage, etc.), σ_b est l'écart-type du bruit de mesure (inconnu en pratique), et $\boldsymbol{x}_0^*$ est l'image à reconstruire.

2 Architecture du projet

2.1 Structure modulaire

Le projet est organisé de manière à séparer clairement les composants :

- `operators_torch.py` – Classe abstraite `LinearOperator` et implémentation `GaussianBlurOperator`. L'utilisateur fournit uniquement `forward(x)` ($\mathcal{A}\boldsymbol{x}$) et `adjoint(y)` ($\mathcal{A}^T\boldsymbol{y}$). Aucune matrice n'est jamais construite ni stockée.
- `emg_vba_1_dim_torch.py` – Algorithme EMG-VBA complet (mises à jour de τ_b, τ_r, distribution de référence, pas sous-optimal, énergie libre négative).
- `diffusion/Reverse.py` – Processus reverse conditionnel : boucle sur $t = T, \ldots, 0$ avec estimation Tweedie, correction EMG-VBA, et pas reverse DDPM.
- `diffusion/Forward.py` – Processus forward et estimateur de Tweedie.
- `diffusion/Schedules.py` – Schedules de bruit (DDPM, compatible diffusers).
- `main_Diffusion_EMG_VBA_1DIM_generique.py` – Script principal générique : l'opérateur, le modèle et les dimensions sont des hyperparamètres en haut du fichier.

2.2 Interface opérateur générique

L'opérateur de dégradation est défini par une classe abstraite :

```
class LinearOperator(ABC):
    def forward(self, x): ...    # A x
    def adjoint(self, y): ...    # A^T y
    def input_dim(self): ...
    def output_dim(self): ...
    def compute_AtA_diag(self): # diag(A^T A)
```

Pour changer d'opérateur de dégradation (flou, inpainting, super-résolution), il suffit d'implémenter `forward` et `adjoint`. Le reste du code (EMG-VBA, reverse, benchmark) est entièrement agnostique.

La méthode `compute_AtA_diag()` calcule $\mathrm{diag}(\mathcal{A}^T\mathcal{A})$ de manière générique via n appels forward + adjoint :

$$[\mathcal{A}^T\mathcal{A}]_{ii} = (\mathcal{A}^T(\mathcal{A}\boldsymbol{e}_i))_i \tag{2}$$

Un opérateur spécifique (par ex. le flou circulant) peut surcharger cette méthode pour exploiter sa structure (FFT).

2.3 Compatibilité multi-modèles

Le script principal détecte automatiquement le type de checkpoint :

— **Modèle diffusers** (ex : `google/ddpm-ema-celebahq-256`) : clés `config` + `alphas_cumprod` dans le `.pt`. Le schedule est reconstruit depuis les $\bar{\alpha}_t$ sauvegardés.

— **Modèle perso** (UNet maison) : clés `model_state_dict` ou `ema_state_dict`. Le schedule est reconstruit depuis les hyperparamètres T, β_{start}, β_{end}.

L'appel réseau est unifié : `output.sample if hasattr(output, 'sample') else output`.

3 Passage CPU → GPU

3.1 Motivation

En passant de 64×64 niveaux de gris ($n = 4\,096$) à 256×256 RGB ($n = 196\,608$), le temps de calcul par itération EMG-VBA augmente d'un facteur ∼48. Sur CPU (numpy, float64), une reconstruction complète prend ∼30 minutes. Le passage en PyTorch sur GPU (MPS / Apple Silicon) réduit ce temps à ∼5 minutes.

3.2 Difficultés rencontrées

1. **Précision numérique.** MPS ne supporte pas `float64`. EMG-VBA en `float32` présentait des oscillations de l'énergie libre causées par un bug de cache (voir ci-dessous), initialement attribuées à tort à la précision numérique.
2. **Bug du cache `_A_mu`.** L'optimisation consistant à cacher $\mathcal{A}\boldsymbol{\mu}$ (calculé dans `mise_a_jour_taur_taub` pour le réutiliser dans `calculer_energie_libre_negative` introduisait une incohérence : après `mise_a_jour_x0`, $\boldsymbol{\mu}$ avait changé mais `_A_mu` n'était pas rafraîchi. L'énergie libre était évaluée avec un $\mathcal{A}\boldsymbol{\mu}$ périmé, causant des oscillations. Le fix : recalculer `_A_mu = op.forward(mu)` à la fin de `mise_a_jour_x0`.
3. **Conversions CPU ↔ GPU.** La version initiale convertissait les tenseurs en numpy à chaque pas t pour EMG-VBA, puis reconvertissait en torch. La version finale garde tout en torch sur le device cible.

4 Pseudo-code de l'algorithme

Algorithm 1 Reverse conditionnel avec correction EMG-VBA

Require: Réseau ϵ_θ, schedule $(\bar{\alpha}_t)_{t=0}^T$, observation $\boldsymbol{y}$, opérateur $\mathcal{A}$
Require: Précalculs : $\mathcal{A}^T\boldsymbol{y}$, $\mathrm{diag}(\mathcal{A}^T\mathcal{A})$

```
x_T ~ N(0, I)
for t = T − 1, ..., 0 do
    Tweedie : x̂_0 ← (x_t − σ_t ε_θ(x_t, t)) / √ᾱ_t
    EMG-VBA : μ_post ← EMG-VBA(x_t, x̂_0, ᾱ_t, y, A)
    Reverse : x_{t−1} ~ q(· | x_t, μ_post)
end for
return x_0
```

Algorithm 2 EMG-VBA à t fixé

Require: $\boldsymbol{x}_t$, $\hat{\boldsymbol{x}}_0$, $\bar{\alpha}_t$, $\boldsymbol{y}$, $\mathcal{A}$, hyperpriors (a_0, b_0, c_0, d_0)

```
Initialiser μ^(0) = x̂_0, Σ^(0) = r_t^2 · 1 avec r_t^2 = (1 − ᾱ_t)/ᾱ_t
for k = 1, ..., K_max do
    Mettre à jour ⟨τ_b⟩, ⟨τ_r⟩                                  (éq. 75–85 de [1])
    Calculer μ^{r,k}, Σ^{r,k} (distribution de référence)        (éq. 94–95)
    Calculer s_1^* = −(∇g)_1 / H_11, clipper à [0, 1]            (éq. 124–138)
    μ^(k) ← interpolation(s_1^*, μ^(k−1), μ^{r,k})               (éq. 115–116)
    Σ^(k) ← interpolation(s_1^*, Σ^(k−1), Σ^{r,k})
    Rafraîchir Aμ^(k) (cache)
    Calculer F(q^(k))                                            (éq. 121–123)
    if |F(q^(k)) − F(q^(k−1))| / |F(q^(k−1))| < ε then
        break (critère d'arrêt adaptatif)
    end if
end for
return μ^(K), Σ^(K), ⟨τ_b⟩, ⟨τ_r⟩
```

5 Équations clés

Nous rappelons ici les équations essentielles implémentées dans le code. Toutes les démonstrations sont dans [1].

5.1 Mise à jour des précisions (éq. 75–85)

$$\tilde{a}_b = m/2 + c_0, \qquad \tilde{b}_b = d_0 + \tfrac{1}{2}\big(\|\boldsymbol{y} - \mathcal{A}\boldsymbol{\mu}\|^2 + \mathrm{diag}(\mathcal{A}^T\mathcal{A})^T\boldsymbol{\Sigma}\big) \tag{3}$$

$$\tilde{a}_r = n/2 + a_0, \qquad \tilde{b}_r = b_0 + \tfrac{1}{2}\big(\|\boldsymbol{\mu} - \hat{\boldsymbol{x}}_0\|^2 + \mathbf{1}^T\boldsymbol{\Sigma}\big) \tag{4}$$

avec $\langle\tau_b\rangle = \tilde{a}_b/\tilde{b}_b$ et $\langle\tau_r\rangle = \tilde{a}_r/\tilde{b}_r$.

5.2 Distribution de référence (éq. 94–95)

$$(\Sigma_i^r)^{-1} = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t} + \langle \tau_b \rangle [\mathcal{A}^T \mathcal{A}]_{ii} + \langle \tau_r \rangle \tag{5}$$

$$\mu_i^r = \Sigma_i^r \left(\frac{\sqrt{\bar{\alpha}_t}}{1-\bar{\alpha}_t} (x_t)_i + \langle \tau_b \rangle [\mathcal{A}^T \boldsymbol{y} - \mathcal{A}^T \mathcal{A} \boldsymbol{\mu} + [\mathcal{A}^T \mathcal{A}]_{ii} \mu_i]_i + \langle \tau_r \rangle (\hat{x}_0)_i \right) \tag{6}$$

5.3 Mise à jour de $q(x_0)$ (éq. 115–116, cas $s_2 = 0$)

$$(\Sigma_i^{k+1})^{-1} = \frac{1-s_1}{\Sigma_i^k} + \frac{s_1}{\Sigma_i^{r,k}} \tag{7}$$

$$\mu_i^{k+1} = \Sigma_i^{k+1} \left(\frac{(1-s_1)\mu_i^k}{\Sigma_i^k} + \frac{s_1 \mu_i^{r,k}}{\Sigma_i^{r,k}} \right) \tag{8}$$

5.4 Énergie libre négative (éq. 121–123)

$$\mathcal{F}(q) = \underbrace{\mathbb{E}_q[\log p(\boldsymbol{w}, \boldsymbol{y})]}_{\text{log-jointe}} - \underbrace{\mathbb{E}_q[\log q(\boldsymbol{w})]}_{\text{entropie croisée}} \tag{9}$$

$\mathcal{F}(q)$ est monotone croissante au cours des itérations EMG-VBA (Résultat XII.2.1 de [1]). C'est une borne inférieure de la log-evidence marginale.

6 Premiers résultats (EMG-VBA seul)

6.1 Configuration expérimentale

Modèle pré-entraîné : `google/ddpm-ema-celebahq-256` (DDPM EMA, CelebA-HQ, 256×256, RGB, $T = 1000$). Opérateur : flou gaussien circulaire ($\sigma_{\text{blur}} \in \{1, 2.5\}$, noyau 9×9). Bruit : $\sigma_b = \sigma_{\text{blur}}/100$. Hyperpriors vagues : $a_0 = b_0 = c_0 = d_0 = 10^{-3}$. Nombre maximal d'itérations EMG-VBA : $K_{\max} = 100$ avec critère d'arrêt adaptatif.

6.2 Reconstruction

Les résultats montrent une reconstruction fidèle pour différentes images (visage, chat, lune) et niveaux de flou. Pour $\sigma_{\text{blur}} = 1$ (flou léger), la reconstruction est quasi indiscernable de l'original (PSNR $\approx$ 28 dB). Pour $\sigma_{\text{blur}} = 2.5$ (flou fort), les détails structurels sont bien récupérés (lunettes, barbe, texture) mais le fond diffère — le modèle génère un fond plausible compatible avec l'observation.

6.3 Convergence de l'énergie libre

L'énergie libre $\mathcal{F}(q)$ est strictement croissante à t fixé, conformément à la théorie. Le critère d'arrêt adaptatif coupe efficacement : $\sim$100 itérations à $t = 999$ (premier pas, initialisation froide), $\sim$7–15 itérations au milieu du reverse (warm start efficace), $\sim$15–20 itérations en fin de reverse.

6.4 Estimation automatique des variances

σ_b^2 estimé ($= 1/\langle\tau_b\rangle$) converge vers l'ordre de grandeur du vrai σ_b^2 : pour $\sigma_{\text{noise}} = 0.01$ ($\sigma_b^2 = 10^{-4}$), l'estimation se stabilise autour de 5×10^{-4}.

σ_r^2 ($= 1/\langle\tau_r\rangle$) décroît régulièrement au cours du reverse : grand aux premiers pas (faible confiance dans l'a priori Tweedie car $\boldsymbol{x}_t$ est très bruité), puis décroît progressivement, et chute vers 0 en fin de reverse (Tweedie quasi exact, l'algorithme s'y colle).

7 Protocole expérimental de comparaison

Nous comparons EMG-VBA à deux méthodes de l'état de l'art pour la résolution de problèmes inverses par diffusion :

7.1 Méthodes comparées

DPS [2]. Diffusion Posterior Sampling. Correction de $\hat{\boldsymbol{x}}_0$ par un gradient de vraisemblance :

$$\boldsymbol{\mu}_{\text{post}} = \hat{\boldsymbol{x}}_0 - \zeta \cdot \mathcal{A}^T(\mathcal{A}\hat{\boldsymbol{x}}_0 - \boldsymbol{y}) \tag{10}$$

Hyperparamètre : ζ (step size, à tuner manuellement).

ΠGDM [3]. Pseudo-Inverse Guided Diffusion Model. Correction par un gradient pondéré :

$$\boldsymbol{\mu}_{\text{post}} = \hat{\boldsymbol{x}}_0 + \eta \cdot \frac{\sqrt{\bar{\alpha}_t}}{\sigma_b^2}\mathcal{A}^T(\boldsymbol{y} - \mathcal{A}\hat{\boldsymbol{x}}_0) \tag{11}$$

Hyperparamètres : η (step size) et σ_b^2 (variance du bruit de mesure, fixée à la vraie valeur).

EMG-VBA (notre méthode). Aucun hyperparamètre. σ_b^2 et r_t^2 sont estimés automatiquement par inférence variationnelle.

7.2 Protocole

1. **Image et dégradation.** Une image de test (256×256, RGB) est dégradée par $\boldsymbol{y} = \mathcal{A}\boldsymbol{x}_0 + \sigma_b\boldsymbol{\varepsilon}$ avec $\sigma_{\text{blur}} = 2.0$, $\sigma_b = 0.02$.
2. **Grille d'hyperparamètres.** DPS : $\zeta \in \{0.1, 0.2, 0.5, 1, 2, 5, 10\}$. ΠGDM : $\eta \in \{0.0005, 0.001, 0.002, 0$ avec $\sigma_b^2 = \sigma_{\text{noise}}^2$ (valeur exacte).

3. **Métriques.** PSNR (dB, ↑), SSIM (↑), cohérence $\|\mathcal{A}\boldsymbol{x}_{\text{rec}} - \boldsymbol{y}\|/\|\boldsymbol{y}\|$ (↓), temps d'exécution.
4. **Même modèle, même observation.** Les trois méthodes utilisent le même réseau pré-entraîné, le même schedule, et la même observation $\boldsymbol{y}$.

Remarque importante : DPS et ΠGDM bénéficient d'un avantage non réaliste : leurs hyperparamètres sont tunés sur le ground truth (impossible en pratique), et ΠGDM utilise la vraie valeur de σ_b^2. EMG-VBA ne dispose d'aucune de ces informations.

8 Résultats du benchmark

8.1 Tableau récapitulatif

Méthode	Hyperparam.	PSNR (dB) ↑	SSIM ↑	Cohérence ↓	Temps (s)
EMG-VBA	**aucun**	**28.30**	**0.759**	**0.043**	336
DPS	$\zeta = 0.1$	21.32	0.552	0.261	225
DPS	$\zeta = 0.5$	28.24	0.833	0.050	224
DPS	$\zeta = 1.0$	29.09	0.853	0.042	225
DPS	$\zeta = 2.0$	29.27	0.861	0.040	225
DPS	$\zeta = 5.0$	<u>29.79</u>	<u>0.862</u>	<u>0.040</u>	225
DPS	$\zeta = 10$	12.74	0.312	0.865	226
ΠGDM	$\eta = 0.001$	29.34	0.860	0.039	225
ΠGDM	$\eta = 0.005$	14.85	0.371	0.673	225
ΠGDM	$\eta \geq 0.01$	< 5	< 0.2	> 2.0	225

Table 1 – Comparaison EMG-VBA / DPS / ΠGDM. Les meilleures valeurs (hors EMG-VBA) sont soulignées. EMG-VBA est en gras.

8.2 Analyse

DPS. Le meilleur ζ (5.0) dépasse légèrement EMG-VBA en PSNR (+1.5 dB) et SSIM. Mais la fenêtre de bon fonctionnement est étroite : $\zeta = 10$ diverge complètement (PSNR = 12.7 dB, artefacts psychédéliques). Le ζ optimal dépend du niveau de bruit et du type d'opérateur — il doit être re-tuné pour chaque configuration expérimentale.

ΠGDM. Encore plus fragile : seule une valeur de η sur 5 donne un résultat correct ($\eta = 0.001$). Un facteur 5 ($\eta = 0.005$) suffit à produire des artefacts colorés massifs. La méthode est inutilisable sans connaissance précise du bon η.

EMG-VBA. Sans aucun tuning et sans connaissance de σ_b, obtient PSNR = 28.3 dB et cohérence = 0.043, ce qui est comparable au meilleur DPS/ΠGDM tunés sur le ground truth. Le surcoût en temps (×1.5) est modeste grâce au critère d'arrêt adaptatif.

Conclusion. Le scatter plot PSNR vs cohérence (Figure **??**) résume l'argument : DPS et ΠGDM forment des courbes (compromis qualité/cohérence dépendant du tuning), tandis qu'EMG-VBA est un point unique en haut à gauche — au bon compromis, automatiquement.

9 Perspectives et idées

9.1 Transport optimal pour l'initialisation inter-pas

Entre deux pas t et $t-1$ du reverse, la loi a posteriori $p_t(x_0|x_t, y)$ se déplace car $\bar{\alpha}_t$, $\boldsymbol{x}_t$ et $\hat{\boldsymbol{x}}_0$ changent. L'initialisation actuelle (warm start naïf : reprendre $\boldsymbol{\mu}_t^*$, $\boldsymbol{\Sigma}_t^*$) ne suit pas ce déplacement.

Idée : estimer le mouvement de la loi a posteriori entre t et $t-1$ via le déplacement de la distribution de référence q_r, puis transporter l'initialisation en conséquence. Pour une distribution gaussienne diagonale, le transport optimal de Wasserstein-2 a une forme fermée coordonnée par coordonnée :

$$\boldsymbol{\mu}_{t-1}^{(0)} = \boldsymbol{\mu}_t^* + (\boldsymbol{\mu}_r^{t-1} - \boldsymbol{\mu}_r^t) \tag{12}$$

$$\boldsymbol{\Sigma}_{t-1}^{(0)} = \boldsymbol{\Sigma}_t^* \cdot \frac{\boldsymbol{\Sigma}_r^{t-1}}{\boldsymbol{\Sigma}_r^t} \tag{13}$$

Coût : un seul appel forward + adjoint pour la nouvelle q_r. Gain espéré : réduire le nombre d'itérations d'un facteur 10–20.

9.2 Sous-espace de dimension 2

L'implémentation actuelle utilise $s_2 = 0$ (pas de mémoire du gradient précédent). Le rapport théorique [1] dérive le cas $s_2 \neq 0$ avec la hessienne 2×2 complète (Résultat XII.5.1). L'ajout de la direction de mémoire devrait atténuer les oscillations de s_1 et accélérer la convergence.

9.3 Preuves de convergence

1. **Monotonie.** $\mathcal{F}(q^{(k+1)}) \geq \mathcal{F}(q^{(k)})$ par construction (le pas s_1 maximise dans un sous-espace).
2. **Bornitude.** $\mathcal{F}(q) \leq \log p(\boldsymbol{x}_t, \boldsymbol{y}) < \infty$, donc la suite converge.
3. **Convergence des paramètres.** À montrer : si $\mathcal{F}(q^{(k)}) - \mathcal{F}(q^{(k-1)}) \to 0$, alors $\|\boldsymbol{\mu}^{(k)} - \boldsymbol{\mu}^{(k-1)}\| \to 0$.
4. **Lien EM.** EMG-VBA est une extension de l'algorithme EM, dont la convergence vers un point stationnaire est établie (Wu, 1983).
5. **Vitesse.** Caractériser le taux de convergence linéaire en fonction de $\bar{\alpha}_t$ (fraction d'information manquante).

9.4 Autres opérateurs de dégradation

Tester la généricité sur : inpainting (masque aléatoire), super-résolution ($\times 4$), et bruit multiplicatif. Seul l'opérateur change — le reste du code est identique.

9.5 Robustesse au niveau de bruit

Varier σ_b sur une large gamme et montrer que EMG-VBA s'adapte automatiquement (σ_b^2 estimé $\approx$ vrai σ_b^2) sans rien changer, alors que DPS/ΠGDM doivent être re-tunés.

9.6 Module Python

Objectif à terme : un package installable (`pip install emgvba-diffusion`) permettant à un utilisateur de résoudre un problème inverse en fournissant uniquement un opérateur (forward/adjoint) et un modèle de diffusion pré-entraîné.

Références

[1] G. Seguy. Rapport théorique – EMG-VBA pour les problèmes inverses par modèles de diffusion. M2 FESup/SIEN, ENS Paris-Saclay, 2026.

[2] H. Chung, J. Kim, M. T. McCann, M. L. Klasky, and J. C. Ye. Diffusion posterior sampling for general noisy inverse problems. In *ICLR*, 2023.

[3] J. Song, A. Vahdat, M. Mardani, and J. Kautz. Pseudoinverse-guided diffusion models for inverse problems. In *ICLR*, 2023.

[4] B. T. Feng, J. Smith, M. Rubinstein, H. Chang, K. L. Bouman, and W. T. Freeman. Score-based diffusion models as principled priors for inverse imaging. In *ICCV*, 2023.

[5] Y. Zheng, A. Fraysse, and T. Rodet. Efficient variational Bayesian approximation method based on subspace optimization. In *IEEE Trans. Image Processing*, 2014.